

PROJECT DOCUMENTATION

Script-Controlled ACL – Restrict Record Access Based on Field Value

ServiceNow Project

1. Introduction

1.1 Project Title

Script-Controlled ACL – Restrict Record Access Based on Field Value

1.2 Team Members and Responsibilities

The project work is divided among the following team members:

- Dhaarani S – ACL Configuration and Script Development
- Swetha S – User, Role and Table Configuration
- Snekhya – ACL Implementation, Functional Testing and Documentation

2. Project Overview

2.1 Purpose

The purpose of the project is to demonstrate how ServiceNow ACLs can be used to enforce controlled access to records. The project combines user roles, table records, field values, and ACL scripts to control READ, CREATE, WRITE, and DELETE operations.

2.2 Key Features

- Creation of a custom Institution Details table.
- Creation of a ServiceNow user and custom roles bb1, bb2, bb3 and bb4.
- Branch-based record access for EEE records.
- Script-controlled READ access.
- Role-controlled CREATE, WRITE and DELETE access.
- Administrator full access.
- Verification through user impersonation.

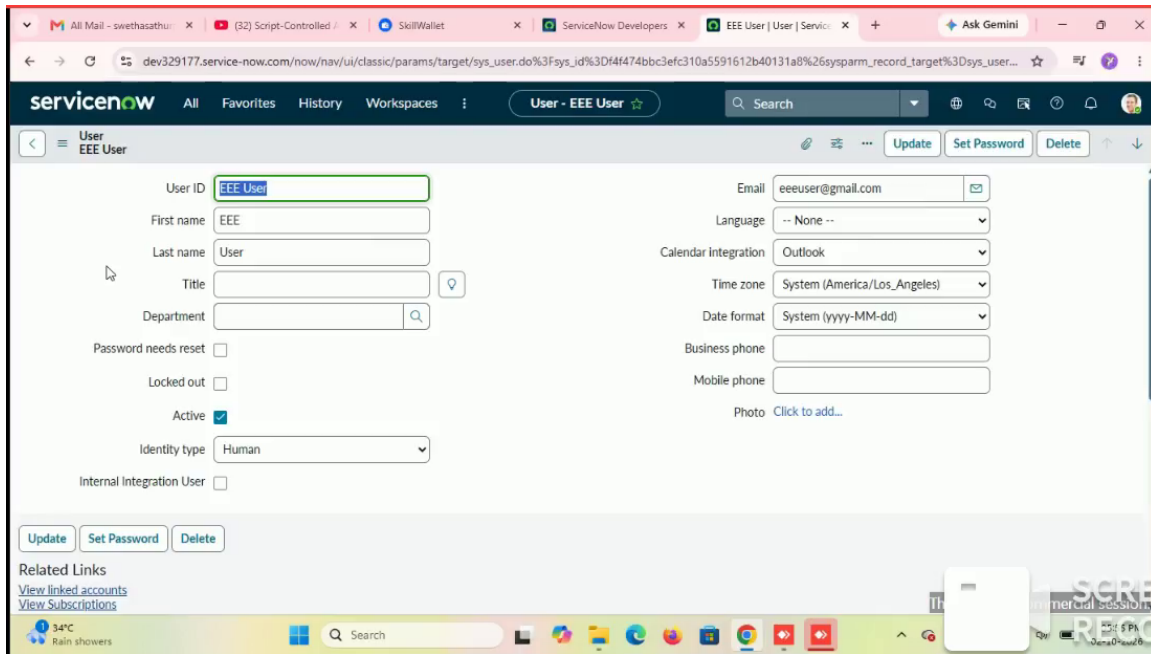


Figure 1: ServiceNow user creation and assigned roles (EEE User).

3. Architecture

3.1 Platform Architecture

Unlike a conventional MERN application, this project is implemented inside the ServiceNow platform. The ServiceNow instance provides the user interface, database tables, role management, ACL engine, and server-side scripting environment.

The logical flow of the implementation is:

- User → ServiceNow interface → ACL evaluation → Role/condition/script evaluation → Institution Details record access.
- Administrator → ACL evaluation → administrative access → permitted operation on records.

3.2 User and Role Layer

The demonstration creates an EEE User and assigns the custom roles bb1, bb2, bb3 and bb4. The roles are used to control the different CRUD operations.

3.3 Data Layer

The custom table is named u_institution_details and is labeled Institution Details. It contains fields for student roll number, student name, faculty name, branch, email, phone number and description.

3.4 Access-Control Layer

The access-control layer contains separate ACL records for READ, CREATE, WRITE and DELETE operations. The READ ACL is advanced and contains a script; the other operations use role requirements as described in the project implementation.

3.5 Logical Architecture Diagram

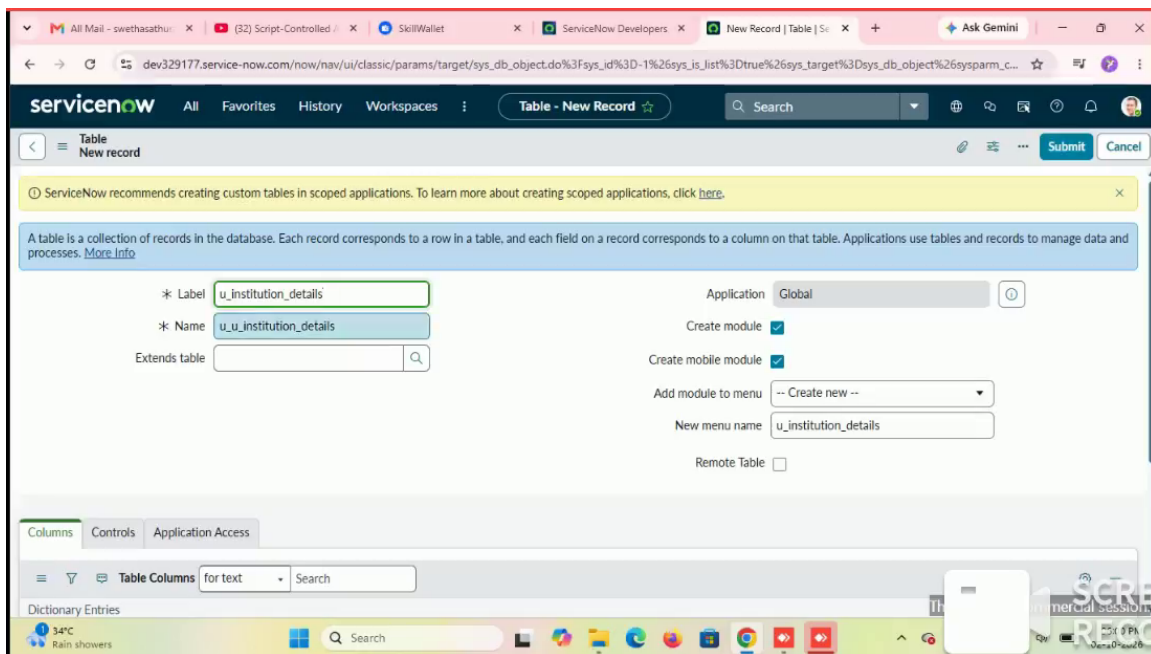
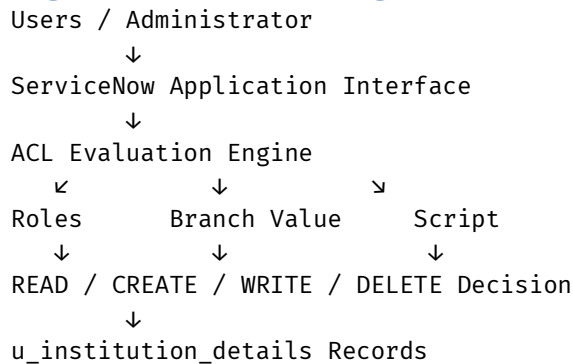


Figure 2: Institution Details table and field configuration.

4. Setup Instructions

4.1 Prerequisites

- Access to a ServiceNow instance with administrator privileges.
- Ability to elevate to the security_admin role for ACL configuration.
- Access to System Definition, User Administration, System Security and related ServiceNow modules.

4.2 User Creation

1. Open User Administration and create a new user.
2. User ID: EEE User.
3. First name: EEE.
4. Last name: User.
5. Email: eeeuser@gmail.com.
6. Create or use the custom roles bb1, bb2, bb3 and bb4 and assign them to the user.

4.3 Custom Table Creation

Create the custom table with the following configuration:

Property	Value
Label	Institution Details
Name	u_institution_details
Extends for	False

4.4 Fields

Field	Type	Purpose
Student Roll Number	Auto Number	Unique student record number
Student Name	Reference – User	References a ServiceNow user
Faculty Name	Reference – User	References a ServiceNow user
Branch	Choice	ECE, EEE, CSE
Email	String	Email value
Phone Number	String	Phone value
Description	Multi String	Record description

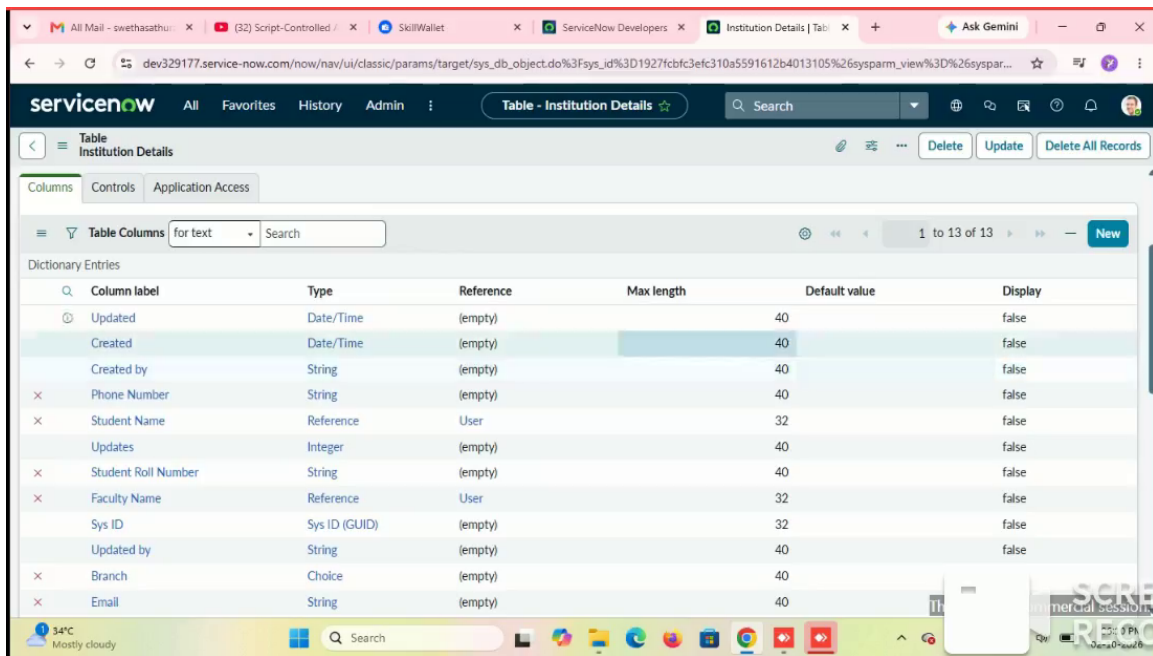


Figure 3: Custom table fields.

5. Project Structure and Configuration

5.1 ServiceNow Configuration Structure

Because this is a ServiceNow configuration project, a conventional client/server folder structure is not applicable. The implementation is organized through ServiceNow configuration records and modules.

- User Administration – EEE User configuration.
- Role records – bb1, bb2, bb3 and bb4.
- System Definition → Tables – Institution Details table.
- System Security → Access Control (ACL) – CRUD ACL records.
- Advanced ACL script – server-side access decision for READ.

5.2 Configuration Summary

Component	Configuration	Purpose
User	EEE User	Demonstration user
Roles	bb1, bb2, bb3, bb4	Control CRUD operations
Table	u_institution_details	Stores institution details
Branch	ECE / EEE / CSE	Record classification
READ ACL	bb1 + script	Controls record visibility

CREATE ACL	bb2	Controls record creation
WRITE ACL	bb3	Controls record modification
DELETE ACL	bb4	Controls record deletion

6. Running the Application / Demonstration Procedure

The project is demonstrated directly in the ServiceNow instance. The following procedure reproduces the access-control workflow.

7. Log in to the ServiceNow instance with administrative privileges.
8. Create the EEE User and the required custom roles.
9. Create the Institution Details table and its fields.
10. Create multiple records with ECE, EEE and CSE branch values.
11. Elevate to security_admin before creating or modifying ACLs.
12. Create the READ ACL and configure its advanced script.
13. Create CREATE, WRITE and DELETE ACLs with the required roles.
14. Impersonate the EEE User and verify the permitted records and operations.
15. Impersonate an administrator and verify full access.

7. API / Server-Side Script Documentation

No REST API endpoints are specified in the supplied project material. The core programmable element documented in the project is the advanced READ ACL script.

7.1 READ ACL Script

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }

    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')) {
        return true;
    }

    // Deny access for all others
    return false;
})();
```

7.2 Script Behaviour

- If the user has the admin role, the script returns true.
- If the user has bb1, the script returns true.
- For users who do not satisfy either role condition, the script returns false.

The project material describes this script as part of the READ ACL scenario in which EEE branch users can view EEE records and administrators retain full access.

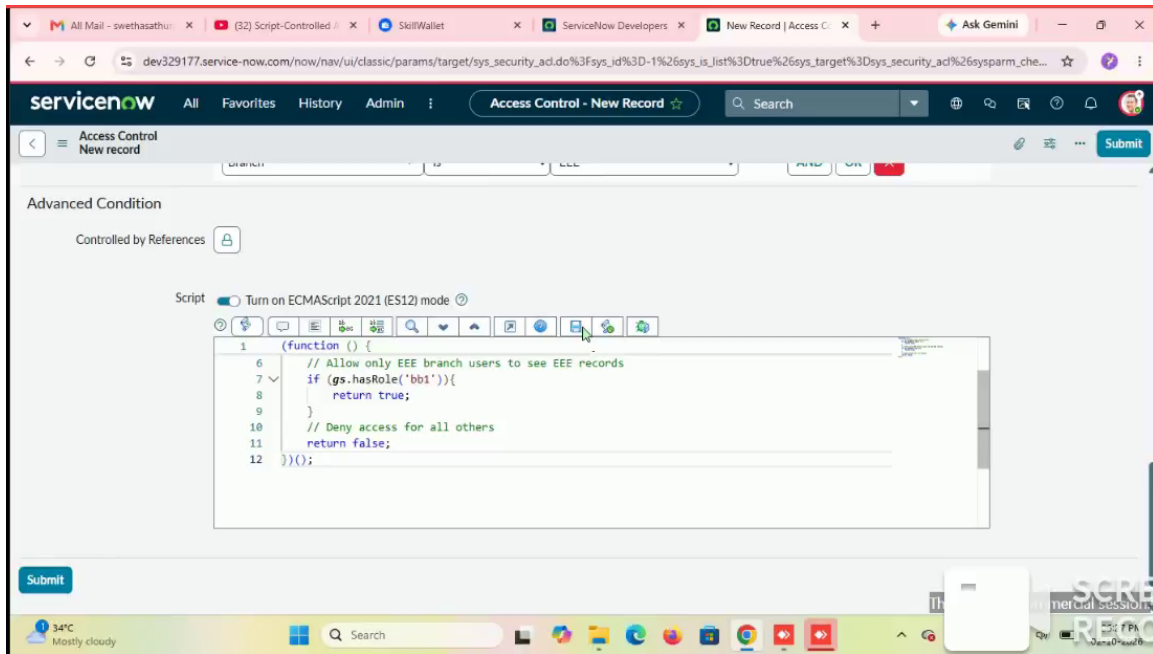


Figure 4: READ ACL configuration and advanced script.

8. Authentication and Authorization

8.1 Authentication

The project uses the authentication facilities of the ServiceNow instance. The supplied implementation does not specify a custom authentication mechanism such as JWT or application-managed sessions.

8.2 Authorization

Authorization is the main focus of the project. ServiceNow roles and ACLs determine whether a user can perform an operation on records.

- bb1 – required for READ ACL access in the demonstrated configuration.
- bb2 – required for CREATE.
- bb3 – required for WRITE.
- bb4 – required for DELETE.
- admin – receives full access in the READ script and retains administrative access.

9. User Interface

The project uses the ServiceNow interface to configure users, roles, tables, records and ACLs. The main user-facing verification is performed through the Student Records list and record forms.

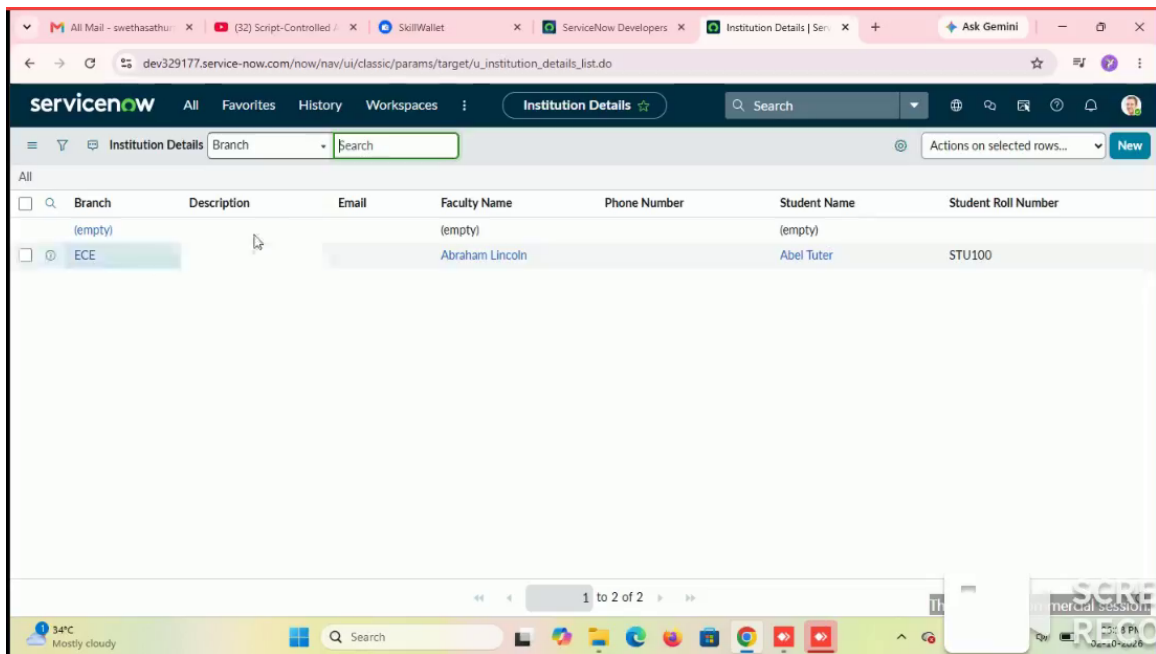


Figure 5: Student Records list for the authorized user.

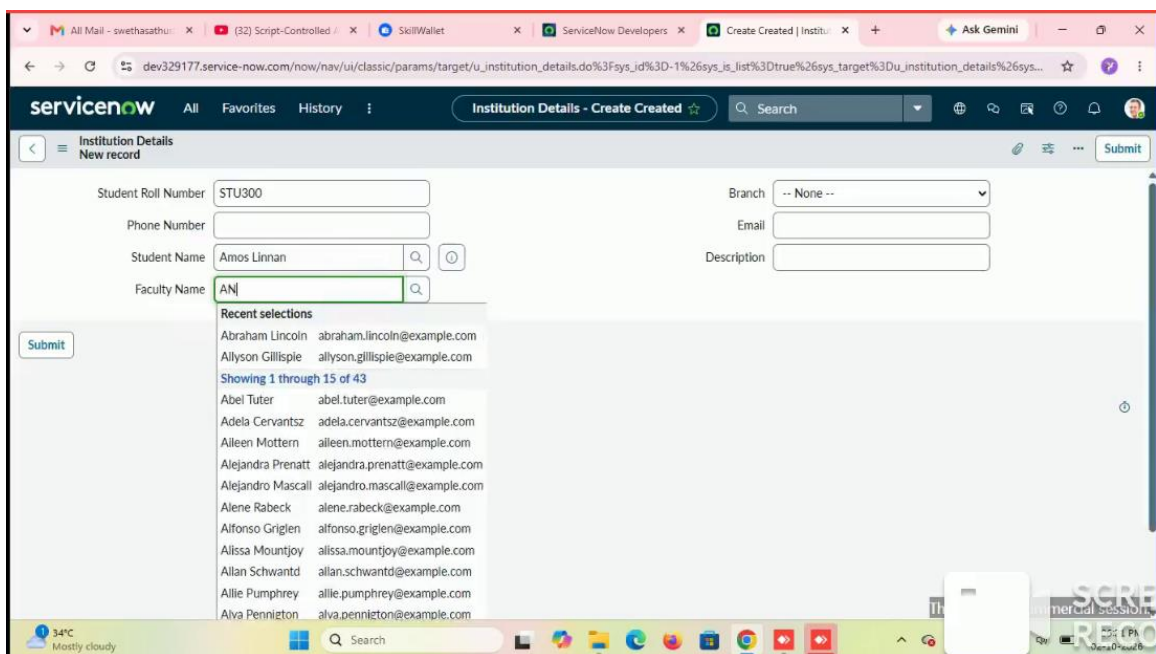


Figure 6: Student record form showing permitted EEE record access.

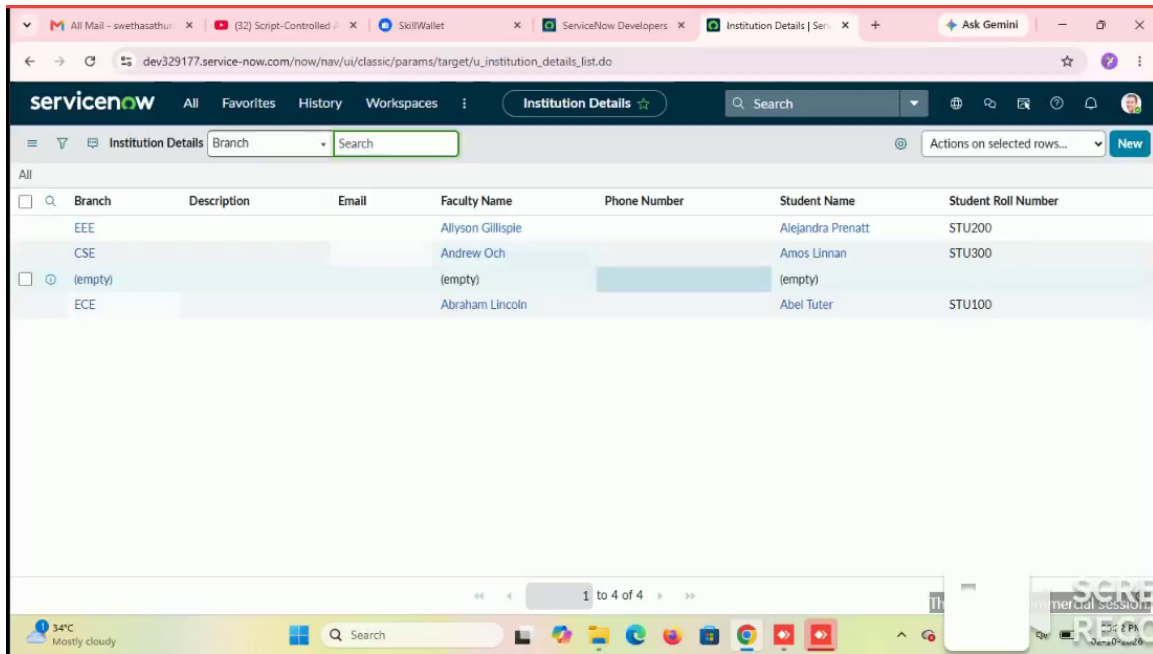


Figure 7: Student Records list for administrator showing full access.

10. Testing

10.1 Testing Strategy

Testing is performed by impersonating users with different role combinations and observing whether the expected CRUD controls are available. The project material verifies the behavior using the ServiceNow Student Records list and form.

10.2 Test Matrix

User / Role Set	Read	Create	Write	Delete	Expected Result
User with bb1	✓	—	—	—	Can view permitted EEE records
bb1 + bb2	✓	✓	—	—	Can view and create
bb1 + bb2 + bb3	✓	✓	✓	—	Can view, create and edit
bb1 + bb2 + bb3 + bb4	✓	✓	✓	✓	Can view, create, edit and delete

Administrator	✓	✓	✓	✓	Full administrative access
---------------	---	---	---	---	----------------------------

10.3 Verification Cases

16. Impersonate EEE User and open the Student Records list.
17. Verify that the intended EEE branch records are visible.
18. Impersonate a user without the required role and verify restricted access.
19. Add bb2 and verify the New button/create capability.
20. Add bb3 and verify edit/write capability.
21. Add bb4 and verify delete capability.
22. Impersonate Admin and verify access to all records.

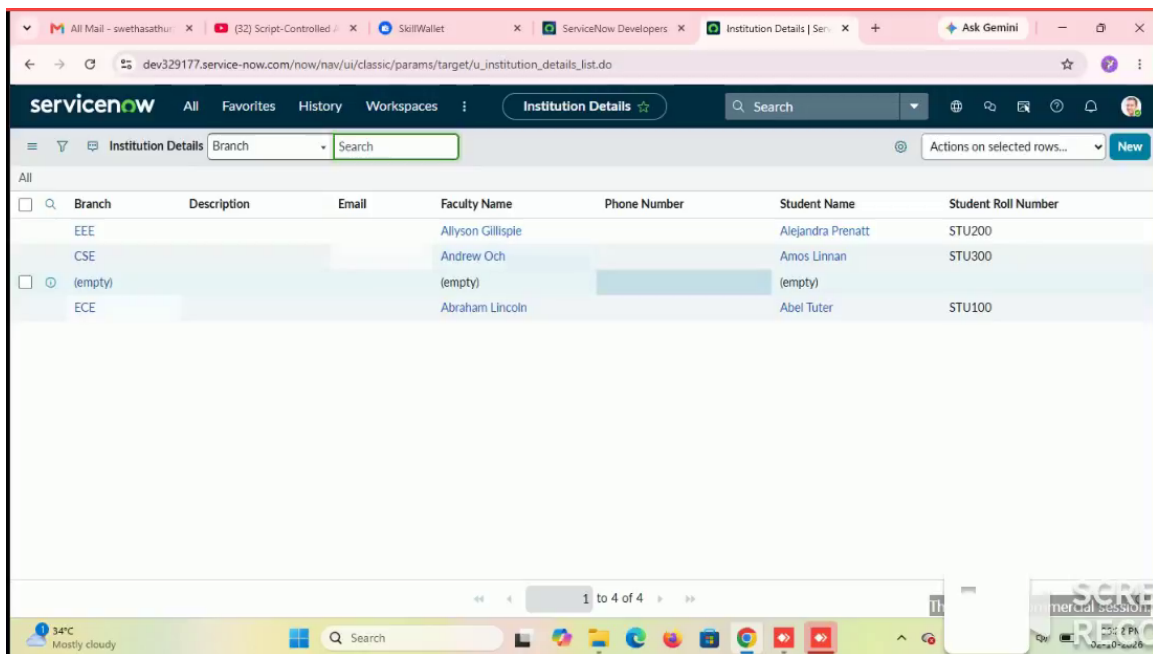


Figure 8: CRUD verification result for authorized user.

11. Screenshots / Demo

The final documentation should contain the actual ServiceNow screenshots in the placeholders provided throughout this report. A live demo link was not supplied.

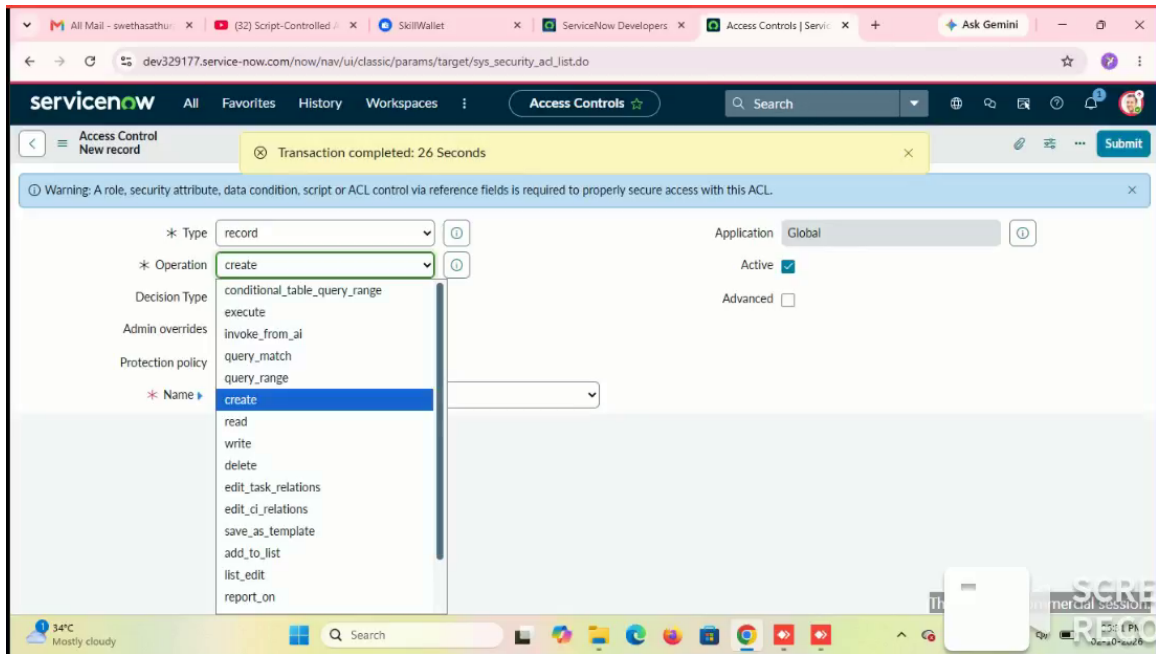


Figure 9: CREATE ACL configuration.

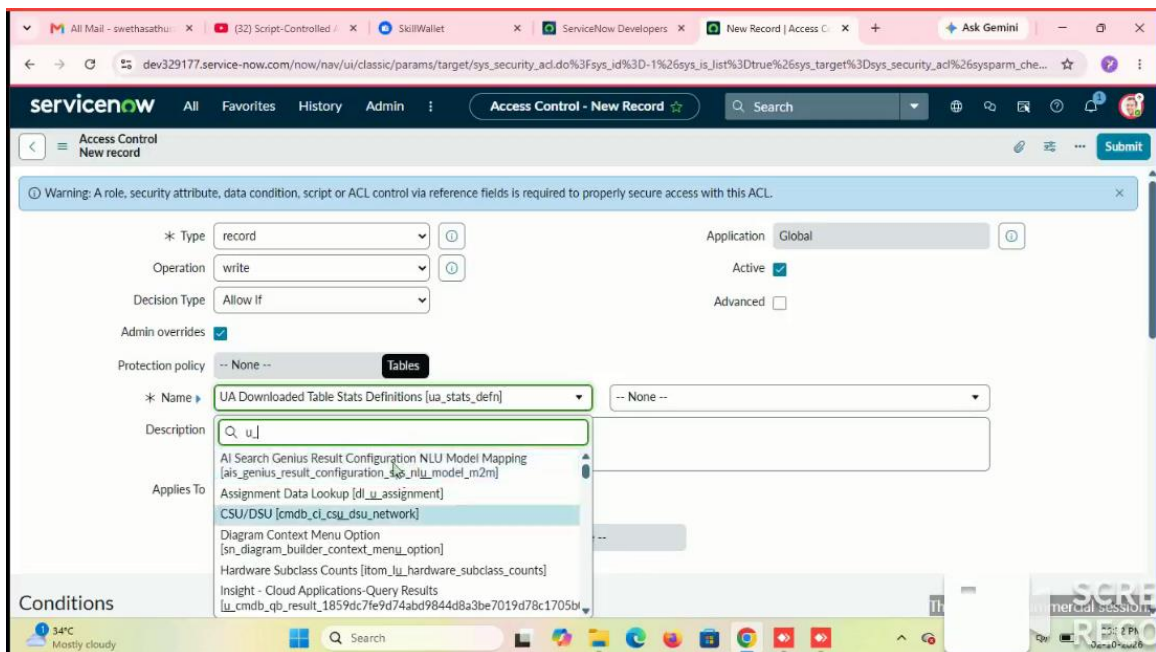


Figure 10: WRITE ACL configuration.

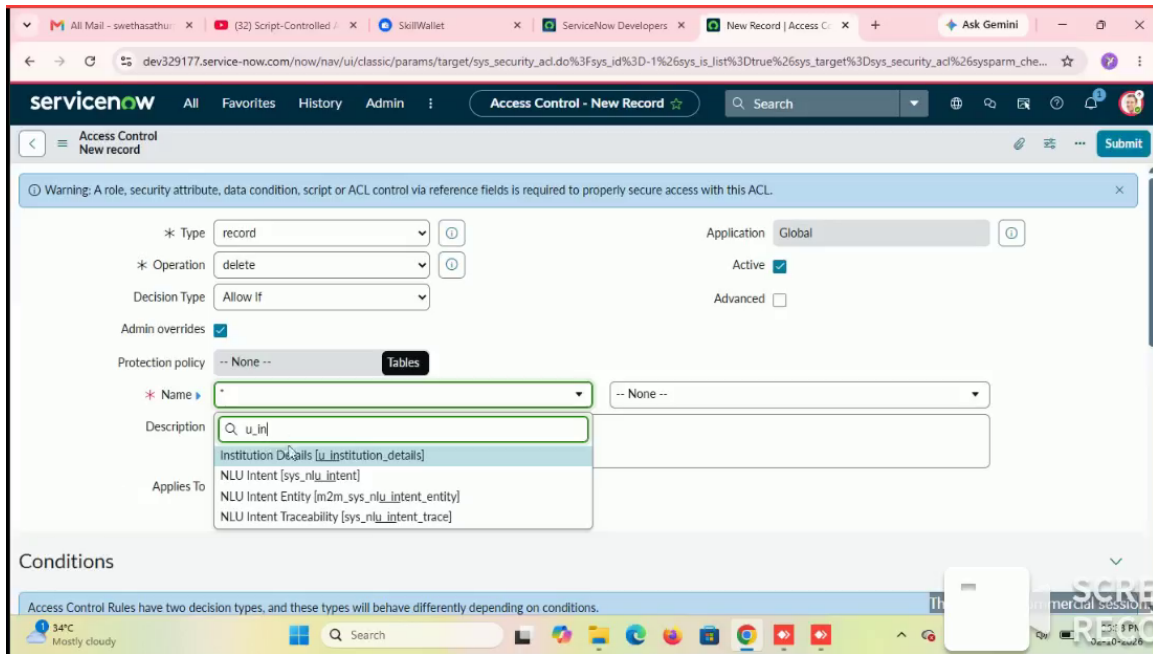


Figure 11: DELETE ACL configuration.

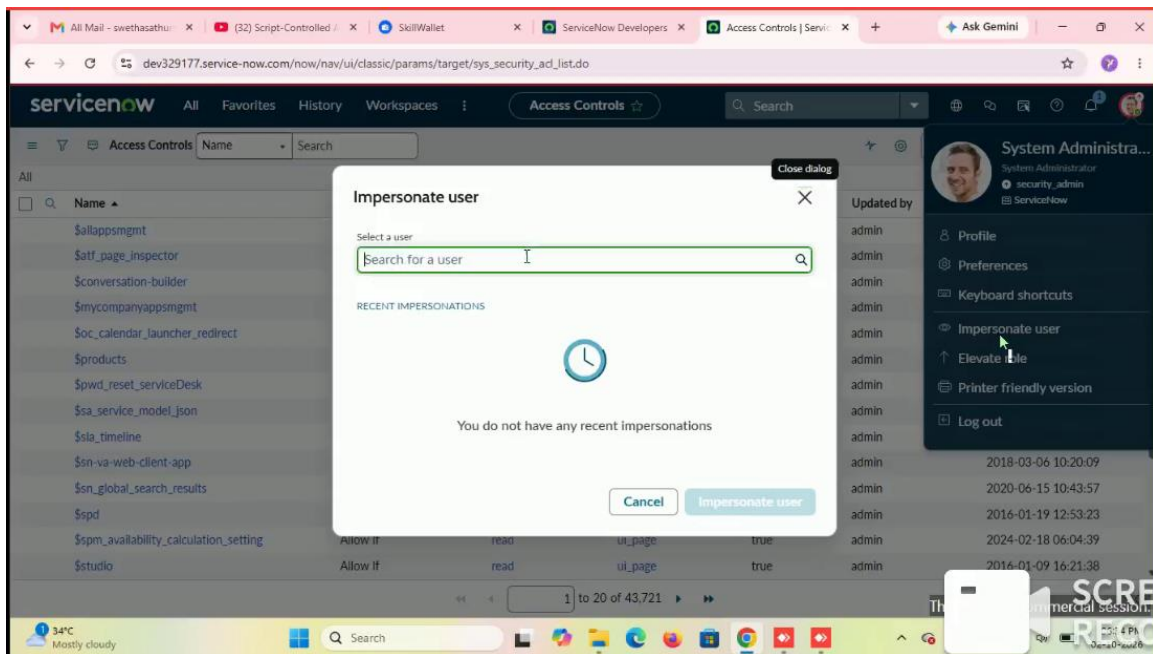


Figure 12: Final CRUD verification / output.

12. Known Issues

The supplied project material does not list a separate set of known software bugs. The following documentation points should therefore be treated as configuration considerations rather than reported defects:

- ACL configuration requires appropriate administrative/security privileges.
- Incorrect role assignment can change the expected access result.
- ACL behavior should be verified after changes by impersonating the relevant users.
- The exact final UI appearance can vary with the ServiceNow instance configuration and version.

13. Future Enhancements

The following are reasonable extensions of the demonstrated ACL configuration. They are proposed enhancements, not features stated as already implemented in the supplied project.

- Extend field-based restrictions to additional branches or departments.
- Introduce more granular roles for different organizational responsibilities.
- Add additional data conditions to protect records based on multiple field values.
- Create automated test cases for ACL behavior.
- Add reporting or dashboards to monitor access-control configuration.
- Document and standardize ACL configuration across multiple custom tables.

14. Conclusion

The project demonstrates how ServiceNow Access Control Lists can be configured to protect records according to user roles and record attributes. A custom Institution Details table is created with branch information, and separate ACLs are configured for READ, CREATE, WRITE and DELETE operations. The implementation shows how role-based authorization and script-controlled access can be combined to restrict unauthorized record operations while preserving administrative access. The verification process through user impersonation provides a practical way to confirm the configured behavior.

15. Appendix

15.1 Project Configuration Details

Custom table: u_institution_details

Roles: bb1, bb2, bb3, bb4

Branch choices: ECE, EEE, CSE

Project platform: ServiceNow

15.2 Links

GitHub: <https://github.com/snekharamiah/Script-Controlled-ACL>

Source Basis

This documentation was prepared from the supplied project reference material describing the Script-Controlled ACL implementation and the supplied documentation format. Where academic metadata, links, or screenshots were not supplied, placeholders have been left for completion.